

AWSKRUG 구디 (Gudi) 챕터 발표

DevSecOps 엔지니어 및 개발자가 AWS 환경에서 안전하고 생산적으로 온보딩하기 위한 체크리스트

신규입사자가 각자의 PC에 세팅하는 항목을 공유합니다 — 보안부터 AI 개발환경, EKS 운영까지

39개 세팅 항목 · **6개** 카테고리 · **1개** 체크리스트

표준화되지 않은 온보딩은 보안과 생산성 모두를 갉아먹습니다

체계 없는 온보딩

- 구두 전달과 사람마다 다른 방식
- 임시 자격증명·VPN 없이 방치되는 보안 공백
- AI 개발도구 연동 여부가 사람마다 제각각
- 실전 투입까지 반복되는 시행착오

체계적인 온보딩 체크리스트

- 보안 → 접근 → 개발 → 운영 순서로 표준화
- MFA 기반 임시 자격증명으로 Zero Trust 유지
- Claude Code·Kiro 등 AI 도구 동일 환경 제공
- 입사 첫 주 안에 실전 투입 가능

두 개의 축으로 정리했습니다

“안전하게” 그리고 “생산적으로”

SEC

안전하게

1. 보안 & ID 계층

Cloudflare WARP · Bitwarden

2. AWS 핵심 접근 계층

AWS CLI · Session Manager · kubectl · MFA 자동화

DEV

생산적으로

3. AI 기반 개발 환경

VS Code+Claude Code · Claude Code Desktop · Kiro

4. 코드 협업 & IaC

Git · GitHub CLI · Terraform · ArgoCD · Vault · Helm · Docker

5. 쿠버네티스 운영 도구

kubectl-argo-rollouts · stern · Freelens

6. 언어·런타임 & 유틸리티

Python·Java·Node·Go · DB/Kafka · 오피스

가장 먼저, 신뢰의 경계부터 세웁니다

온보딩 첫날 세팅하는 최소한의 Zero Trust 기반

VPN

Cloudflare WARP

사내망 접근을 위한 Zero Trust VPN. 디바이스 인증 기반으로 접속을 통제합니다.

PW

Bitwarden

자주 쓰는 개인 비밀번호를 저장해두고 편하게 꺼내 쓰는 개인용 관리자. DB·API 등 핵심 시크릿은 Vault에서 별도 관리합니다.

MFA

AWS MFA 임시 자격증명

매일 아침 MFA 인증 한 번으로 24시간 유효한 임시 자격증명을 발급받습니다. (다음 장에서 상세 설명)

Zero Trust 3단 방어

1

네트워크

Cloudflare WARP로 사내망 진입

2

자격증명 분리

개인 비밀번호는 Bitwarden, 핵심 시크릿은 Vault로 이원화

3

AWS 권한

MFA 기반 임시 자격증명(24h)

WHY 개인 비밀번호와 회사의 핵심 시크릿을 같은 곳에 섞어두면 사고 범위가 커집니다. Bitwarden(개인용)과 Vault(DB·OAuth·API 키)를 분리해 관리합니다.

AWS 인프라에 안전하게 닿는 법

SSH 대신 IAM 기반 접근, 콘솔 대신 CLI를 우선합니다



AWS CLI (aws configure)

SSO/Named Profile 기반으로 계정·리전을 구성합니다.



AWS Session Manager

SSH 포트 없이 IAM 권한으로 EC2/베스천에 접속합니다.



kubectl (kube config)

EKS 클러스터 컨텍스트를 등록해 바로 운영에 투입합니다.



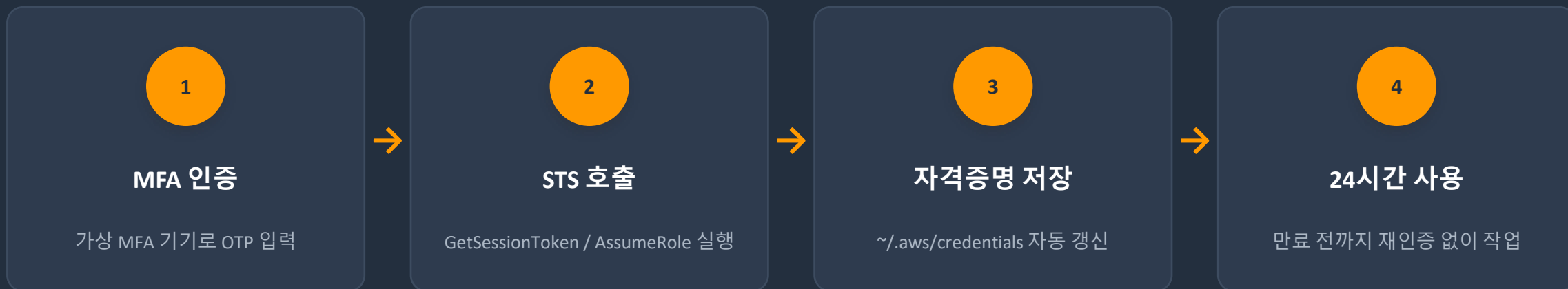
MFA 임시 자격증명 자동화

AssumeRole 기반 24시간 임시 크리덴셜을 자동 발급합니다.

WHY SSH 포트를 열어두는 것 자체가 공격 표면입니다. Session Manager는 포트 없이 IAM 권한만으로 접속을 통제해, 베스천 운영과 키 관리 부담을 함께 없애줍니다.

MFA 기반 임시 보안 자격증명, 자동화하기

보안팀 정책(MFA 필수)과 개발자 생산성을 동시에 만족시키는 스크립트



Why: 장기 Access Key 상시 노출을 없애고, 매일 MFA 인증을 강제해 Zero Trust 원칙을 지키면서도 하루 한 번 인증으로 반복 로그인 피로를 줄입니다.

코드를 작성하는 방식 자체가 달라졌습니다

IDE부터 데스크톱 에이전트까지, AI를 온보딩 표준에 포함합니다

VSC

VS Code + Claude Code

코드 작성용 메인 IDE

- Claude Code for VS Code
- GitHub Actions
- GitLens
- HashiCorp Terraform
- Helm Intellisense
- Python / YAML

AGT

Claude Code Desktop

업무 전반을 넘나드는 에이전트

- Datadog
- Google Drive
- Slack
- Jira
- PPT
- Chrome Browser

KRO

Kiro IDE

AWS 네이티브 AI 개발환경

- Agent Toolkit for AWS 연동
- AWS 리소스 맥락 기반 코드 생성
- 인프라 변경 제안 및 리뷰

WHY 온보딩 첫 주의 가장 큰 병목은 코드가 아니라 '사내 컨벤션 파악'입니다. Claude Code·Kiro로 코드베이스 맥락을 빠르게 캐치하면 실전 투입 시점이 앞당겨집니다.

코드와 인프라, 같은 방식으로 다룹니다

버전관리부터 배포·시크릿까지 이어지는 워크플로우

GIT

Git (SSH Key 연동)

패스워드 없이 SSH 키로 원격 저장소 인증

GH

GitHub CLI

터미널에서 PR·이슈·리뷰를 바로 처리

TF

Terraform

AWS 인프라를 코드로 선언하고 관리

AR

ArgoCD CLI

GitOps 기반 애플리케이션 배포·동기화

VLT

Vault CLI

DB·OAuth·API 등 핵심 시크릿을 중앙 관리

H**M**

Helm CLI

쿠버네티스 애플리케이션 패키지·배포 관리

DK

Docker Desktop → Harbor

참고 · AWS ECR 미사용

로컬에서 이미지를 빌드하고, AWS ECR 대신 사내 Harbor 레지스트리로 바로 푸시합니다. 오늘 스택 중 AWS 서비스 대신 사내 인프라를 쓰는 유일한 지점이에요.

EKS 클러스터를 다루는 손에 익은 도구들

배포 확인부터 로그 추적까지, 터미널 위주 워크플로우

ROL

kubectl-argo-rollouts

카나리/블루그린 배포 진행 상황을 CLI에서 실시간으로 확인합니다.

LOG

stern

여러 Pod의 로그를 라벨 기준으로 동시에 tail 합니다.

LNS

Freelens

클러스터 리소스를 시각적으로 살펴보는 오픈소스 K8s GUI.

WHY 파드 로그를 하나씩 `kubectl logs`로 확인하다 장애 대응이 늦어지는 경우가 흔합니다. `stern`은 여러 Pod를 한 화면에 모아 이 지연을 없앱니다.

생산성 부스터

PowerShell Alias

```
function k { kubectl @args }
```

```
Set-Alias k kubectl
```

긴 `kubectl` 명령을 `k` 한 글자로 줄여 매일 수십 번의 타이핑을 절약합니다.

실행 환경과 로컬 데이터 스택을 맞춥니다

여러 서비스가 쓰는 언어와 로컬 개발용 DB/메시징까지

언어 & 런타임

PY

Python

JV

Java (OpenJDK)

JS

Node.js

GO

Go

로컬 데이터 스택

KFK

Kafka 로컬
메시징 파이프라인 테스트

PG

PostgreSQL
로컬 개발 DB

DBV

DBeaver
범용 DB GUI 클라이언트

WHY 팀마다 다른 런타임 버전을 로컬에 깔면 '내 PC에서는 되는데요' 문제가 반복됩니다. 로컬 Kafka·DB로 미리 검증하면 이런 재현 불가 버그를 줄일 수 있습니다.

매일 손에 쥐는 자잘하지만 필수적인 도구들

장애 대응 중 로그 파싱·화면 캡처가 늦어지는 건 대부분 이런 사소한 도구가 없어서입니다

jq

curl

ncat

Notepad++

에스캡처

Adobe PDF Reader

MS PowerToys

Datadog CLI

오피스 & 협업

MS Office

문서·스프레드시트 작업

Slack

팀 커뮤니케이션

Chrome Browser

웹 콘솔·SaaS 도구

Email 서명

회사 표준 서명 템플릿 적용

온보딩 체크리스트 한눈에 보기

39개 항목 · 6개 카테고리 — 이 표를 그대로 우리 팀 온보딩 문서에 옮겨보세요

보안 & ID

- ✓ Cloudflare WARP
- ✓ Bitwarden
- ✓ MFA 자동화 스크립트

AWS 핵심 접근

- ✓ AWS CLI
- ✓ Session Manager
- ✓ kubectl
- ✓ MFA 자동화

AI 개발 환경

- ✓ VS Code+Claude Code
- ✓ Claude Code Desktop
- ✓ Kiro+Agent Toolkit

코드 협업 · IaC

- ✓ Git (SSH)
- ✓ GitHub CLI
- ✓ Terraform
- ✓ ArgoCD CLI
- ✓ Vault CLI
- ✓ Helm CLI
- ✓ Docker Desktop

K8s 운영

- ✓ kubectl-argo-rollouts
- ✓ stern
- ✓ Freelens
- ✓ PowerShell alias

런타임 · 데이터

- ✓ Python / Java / Node / Go
- ✓ Kafka 로컬
- ✓ DBeaver / PostgreSQL

프론트엔드 & Cloud 개발자의 세팅

소소한 일반적인 세팅 팁부터, 개발까지 나아가는 세팅 방법

COM

일반적인

1. 계정 관리

Google · Jira · Confluence

2. 유틸리티

주로 사용하는 Application

DEV

개발용

3. 개발 환경 세팅

Node · VS Code

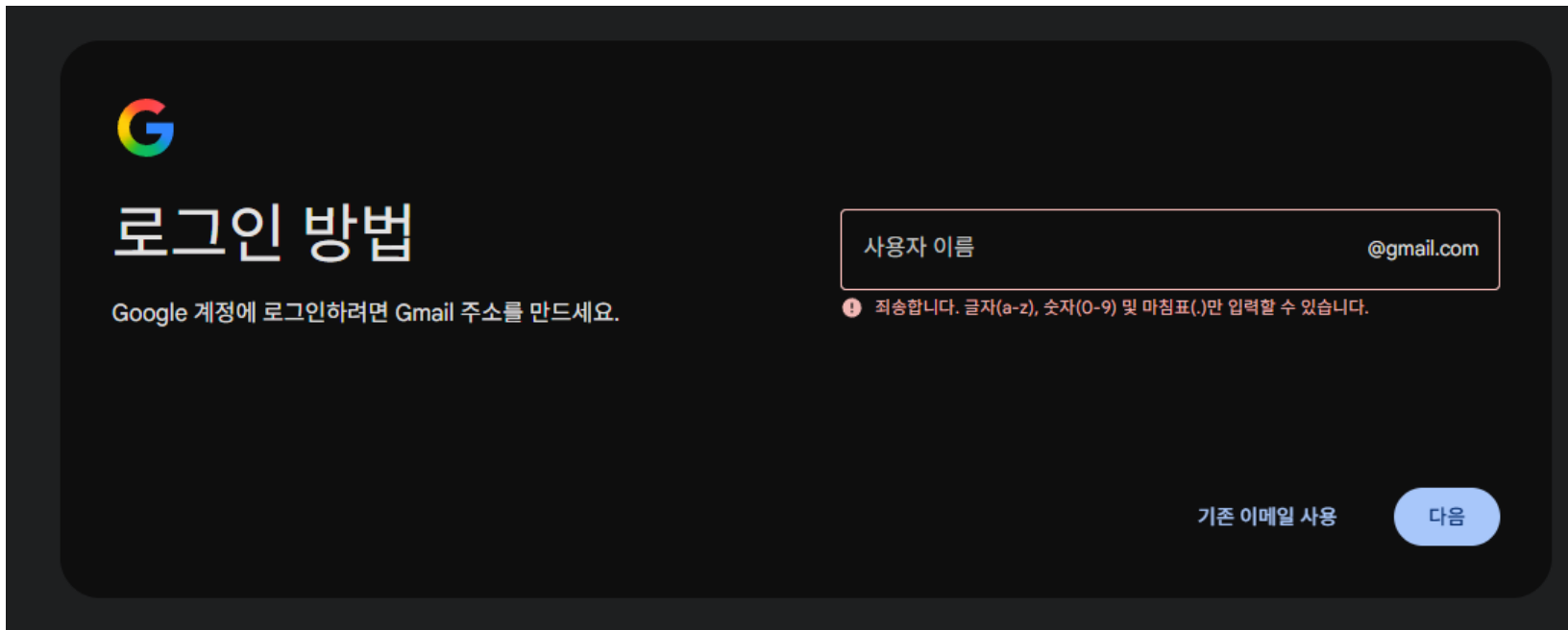
4. 코드 협업

Git · GitHub CLI

5. AI 기반 도구

Claude Code

다른 계정으로 ID 분리





로그인 방법

Google 계정에 로그인하려면 Gmail 주소를 만드세요.

사용자 이름 @gmail.com

 죄송합니다. 글자(a-z), 숫자(0-9) 및 마침표(.)만 입력할 수 있습니다.

기존 이메일 사용 다음

기존 이메일 사용으로,
회사용 과 개인용 credential(id/password etc..) 분리

없으면 허전한 유틸리티 툴

WSL2

Postman

Alfred(Mac)

Notepad++

ScreenToGif

Adobe PDF Reader

OrbStack(Mac)

반디집

오피스 & 협업

MS Office

문서·스프레드시트 작업

Slack

팀 커뮤니케이션

Chrome Browser

웹 콘솔·SaaS 도구

Figma

기획서 & 디자인 시안 열람

개발을 쉽게 하기 위한 세팅

VSCode 어디까지 세팅하셨나요 다들?

1

편의성

반복 일을 단축해주는 Extension

- Auto Close Tag
- Hungry Delete
- Path Intellisense
- Tabout
- JavaScript code snippet

2

컨벤션

Lint & Code formatting

- ESLint
- Gitmoji
- Prettier
- Todo Tree

3

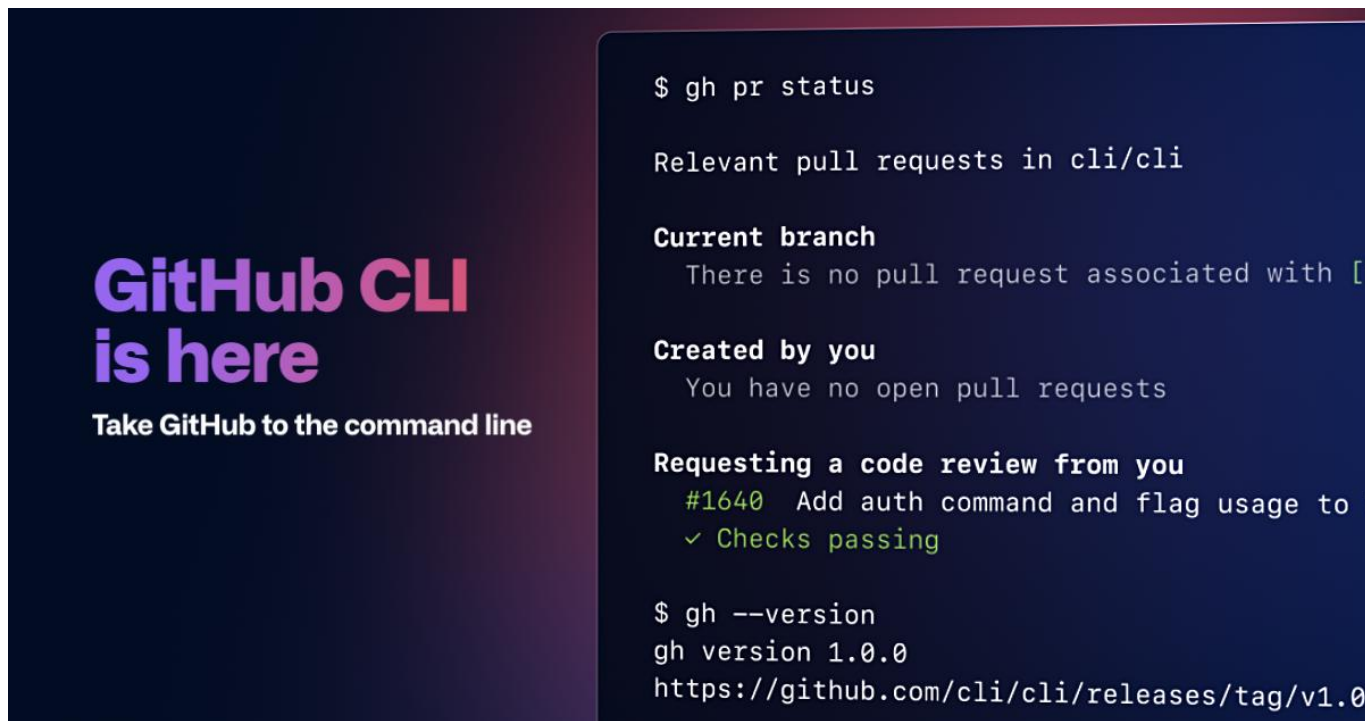
UI

나만의 익숙한 UI

- Night Owl
- Fira Code Nerd Font
- Material Icon Theme
- Markdown Preview
- DotENV
- Rainbow CSV
- Git Graph

Github CLI를 설치해야 하는 이유

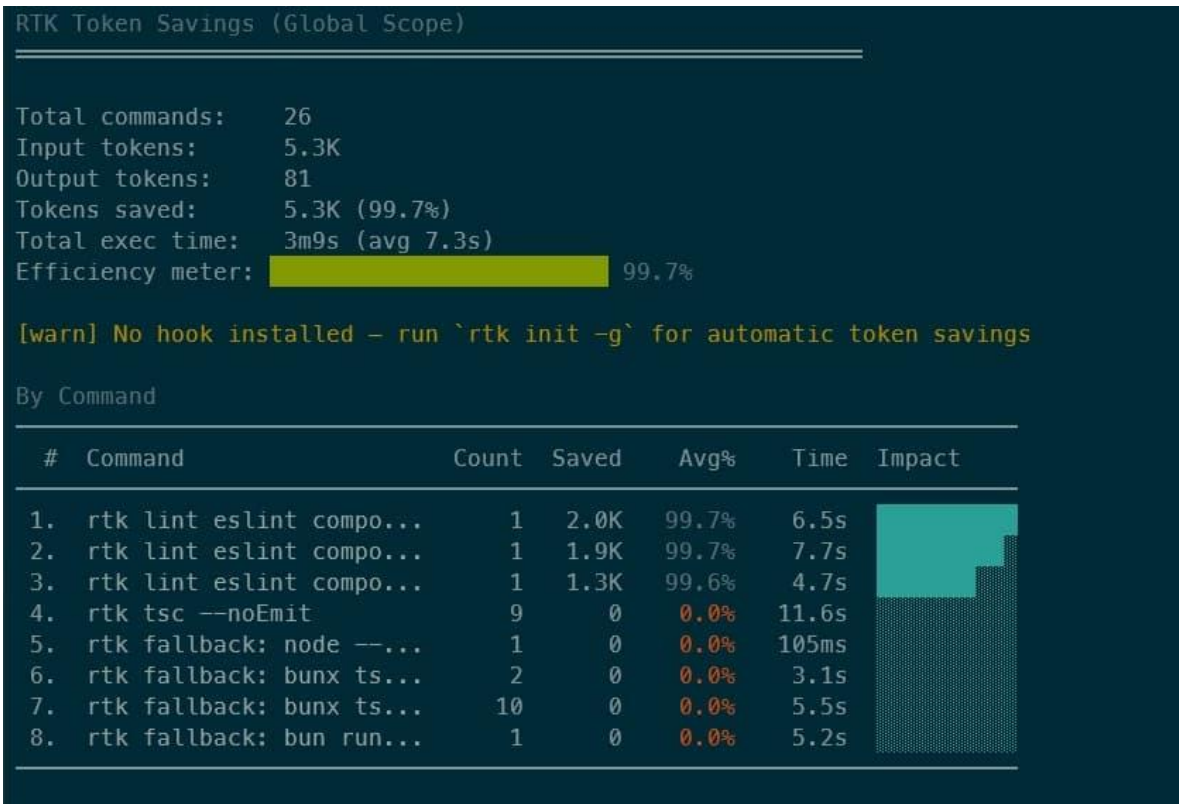
AI 환경에서 협업하기 위한 조건



전통적인 방식도 있지만,
AI 활용하는 조직에서는 필수

토큰 사용량을 줄여봅시다.

RTK를 사용한 토큰 줄이기



- Prehook 기반 Token Saver
- Output을 위주로 줄여줌

AI를 너무 신뢰하지 않습니다.

AI에게 최소한의 안전장치를 걸어놓기

```
#!/usr/bin/env bash
# PreToolUse(Bash) guard: block creative/destructive AWS CLI calls.
# Policy: only read-only AWS calls (list/get/describe/...) are allowed.
# See .claude/memory/aws-readonly-only.md
set -euo pipefail

cmd="$(jq -r '.tool_input.command // ""')"
```

Fast path: nothing to do if there's no aws invocation.

```
if ! printf '%s' "$cmd" | grep -qE '^[^[:alnum:]]aws([[:alnum:]]|$)'; then
    exit 0
fi
```

deny() {

```
    jq -nc --arg r "$1" \
      '{hookSpecificOutput:{hookEventName:"PreToolUse",permissionDecision:"deny",permissionDecisionReason:$r}}'
```

exit 0

}

Destructive aws operation verbs.

A: blocked whether bare or hyphenated (unlikely to appear as shell words/args)

B: blocked only in hyphenated aws-operation form (also common shell words)

```
verbsA='create|delete|terminate|deregister|modify|reboot|purge|revoke|disassociate|associate|authorize|untag|provision'
verbsB='put|update|remove|register|enable|disable|cancel|reset|restore|import|export|replace|release|request|accept|reject|apply|execute|inv$'
```

```
prev=""
for tok in $cmd; do
    # s3 mutating subcommands: aws s3 cp|mv|rm|sync|mb|rb
    if [[ "$prev" == "s3" && "$tok" =~ ^(cp|mv|rm|sync|mb|rb)$ ]]; then
        deny "AWS read-only 정책: 'aws s3 $tok' 변경 쿼리는 차단됨. list/get/describe만 허용."
```

fi

```
    if [[ "$tok" =~ ^(${verbsA})-([a-z0-9-]+)?$ ]]; then
        deny "AWS read-only 정책: '$tok' 생성/변경/삭제성 쿼리는 차단됨. list/get/describe만 허용."
```

fi

```
    if [[ "$tok" =~ ^(${verbsB})-([a-z0-9-]+)$ ]]; then
        deny "AWS read-only 정책: '$tok' 생성/변경/삭제성 쿼리는 차단됨. list/get/describe만 허용."
```

fi

```
    prev="$tok"
done
exit 0
```

감사합니다

Q&A · 오늘 나온 세팅 항목은 정리해서 공유드리겠습니다

AWSKRUG 구디 (Gudi) 챗터